Lab Final

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

1 Lab Final

1. Calculate cost per minute for films, sorted in Descending order.

```sql
SELECT
    title,
    replacement_cost / length AS cost_per_minute
FROM film
WHERE length > 0
ORDER BY cost_per_minute DESC;
```

2. Write a query that returns category name and average film length for categories whose average film length is greater than the overall average film length.

```sql
SELECT cat.name,
       AVG(f.length) AS avg_length
FROM category cat
JOIN film_category fc ON fc.category_id = cat.category_id
JOIN film f ON f.film_id = fc.film_id
GROUP BY cat.name
HAVING AVG(f.length) >
       (SELECT AVG(length) FROM film)
ORDER BY avg_length DESC;
```

3. Find customers who have rented the same film more than once. Show customer id, full name, film title, and rental count.

```sql
SELECT c.customer_id,
       c.first_name,
       c.last_name,
       f.title,
       COUNT(*) AS rental_count
FROM customer c
JOIN rental r ON r.customer_id = c.customer_id
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f ON f.film_id = i.film_id
GROUP BY c.customer_id, c.first_name, c.last_name, f.title
HAVING COUNT(*) > 1
ORDER BY rental_count DESC, c.customer_id;
```

4. Find actors whose films span all available rating categories. Show actor id and full name.

```sql
SELECT a.actor_id,
       a.first_name,
       a.last_name
FROM actor a
JOIN film_actor fa ON fa.actor_id = a.actor_id
JOIN film f ON f.film_id = fa.film_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(DISTINCT f.rating) =
       (SELECT COUNT(DISTINCT rating) FROM film)
ORDER BY a.actor_id;
```

5. For each city, calculate the total revenue generated by customers from that city sorted in descending order.

```
SELECT ci.city, co.country, COALESCE(SUM(p.amount),0) AS revenue
FROM city ci
JOIN address a ON ci.city_id = a.city_id
JOIN customer c ON a.address_id = c.address_id
JOIN payment p ON c.customer_id = p.customer_id
JOIN country co ON ci.country_id = co.country_id
GROUP BY ci.city, co.country
ORDER BY revenue DESC;
```

6. Write an anonymous block that prints the top 5 categories that contain the most films. Print category id, name, and film count using a loop.

```
DO $$
DECLARE
    rec RECORD;
    i INT := 0;
BEGIN
    FOR rec IN
        SELECT c.category_id, c.name, COUNT(fc.film_id) AS film_count
        FROM category c
        JOIN film_category fc ON fc.category_id = c.category_id
        GROUP BY c.category_id, c.name
        ORDER BY film_count DESC, c.category_id
    LOOP
        i := i + 1;
        EXIT WHEN i > 5;

        RAISE NOTICE '%) Category %: % (Films=%)',
            i, rec.category_id, rec.name, rec.film_count;
    END LOOP;
END $$;
```

7. Write an anonymous block that takes an actor id (e.g., 1) and prints all film titles that the actor appeared in. Print actor name once at the start.

```
DO $$
DECLARE
    v_actor_id INT := 1;
    v_first TEXT;
    v_last TEXT;
    rec RECORD;
BEGIN
    SELECT first_name, last_name
    INTO v_first, v_last
    FROM actor
    WHERE actor_id = v_actor_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Actor % not found', v_actor_id;
        RETURN;
    END IF;

    RAISE NOTICE 'Films for Actor % % (id=%):', v_first, v_last, v_actor_id;

    FOR rec IN
        SELECT f.title
```

```
        FROM film f
        JOIN film_actor fa ON fa.film_id = f.film_id
        WHERE fa.actor_id = v_actor_id
        ORDER BY f.title
    LOOP
        RAISE NOTICE ' - %', rec.title;
    END LOOP;
END $$;
```

8. Create a procedure that inserts a payment for that rental using the same customer_id and staff_id from `rental` and uses today's date. If `p_amount <= 0` or if the rental id does not exist, raise an exception.

```
CREATE OR REPLACE PROCEDURE pay_fine(p_rental_id INT, p_amount NUMERIC)
LANGUAGE plpgsql
AS $$
DECLARE
    v_customer_id INT;
    v_staff_id INT;
BEGIN
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'Amount must be positive';
    END IF;

    SELECT customer_id, staff_id
    INTO v_customer_id, v_staff_id
    FROM rental
    WHERE rental_id = p_rental_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Rental id % not found', p_rental_id;
    END IF;

    INSERT INTO payment(customer_id, staff_id, rental_id, amount, payment_date)
    VALUES (v_customer_id, v_staff_id, p_rental_id, p_amount, NOW());

    RAISE NOTICE 'Payment inserted for rental % (amount=%)', p_rental_id,
    p_amount;
END;
$$;

-- Invocation (example)
CALL pay_fine(1, 3.50);
```

9. Create a function that returns the actor pair (actor1_id, actor2_id) that appeared together in the most films and the film_count.

```
CREATE OR REPLACE FUNCTION most_frequent_actor_pair()
RETURNS TABLE (
    actor1_id SMALLINT,
    actor2_id SMALLINT,
    film_count INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
```

```
    SELECT fa1.actor_id,
           fa2.actor_id,
           COUNT(*)::INT
    FROM film_actor fa1
    JOIN film_actor fa2
      ON fa1.film_id = fa2.film_id
     AND fa1.actor_id < fa2.actor_id
    GROUP BY fa1.actor_id, fa2.actor_id
    ORDER BY COUNT(*) DESC, fa1.actor_id, fa2.actor_id
    LIMIT 1;
END;
$$;

-- Example:
SELECT * FROM most_frequent_actor_pair();
```

10. Write a procedure that determines whether a film is above average in rentals by comparing its rental count to the average rental count per film.

```
CREATE OR REPLACE PROCEDURE check_film_popularity_proc(p_film_id INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_title TEXT;
    v_count INT;
    v_avg NUMERIC;
BEGIN
    -- Get film title
    SELECT title
    INTO v_title
    FROM film
    WHERE film_id = p_film_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Film id % not found', p_film_id;
        RETURN;
    END IF;

    -- Rental count for this film
    SELECT COUNT(r.rental_id)
    INTO v_count
    FROM inventory i
    JOIN rental r ON r.inventory_id = i.inventory_id
    WHERE i.film_id = p_film_id;

    -- Average rental count per film
    SELECT AVG(cnt)
    INTO v_avg
    FROM (
        SELECT COUNT(r2.rental_id) AS cnt
        FROM inventory i2
        JOIN rental r2
            ON r2.inventory_id = i2.inventory_id
        GROUP BY i2.film_id
    ) sub;

    -- Compare and print result
    IF v_count > v_avg THEN
```

```
        RAISE NOTICE
        'Film: % | Rentals: % | Avg: % -> ABOVE average',
        v_title, v_count, ROUND(v_avg, 2);
    ELSE
        RAISE NOTICE
        'Film: % | Rentals: % | Avg: % -> NOT above average',
        v_title, v_count, ROUND(v_avg, 2);
    END IF;
END;
$$;

CALL check_film_popularity_proc(1);
```

2 ER Diagram


actor
actor_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
last_update DATETIME *
language
language_id INT * pk
name CHAR(20) *
last_update DATETIME *
film
film_id INT * pk
title VARCHAR(255) *
description TEXT
release_year YEAR
language_id INT *
rental_duration SMALLINT *
rental_rate DECIMAL(4,2) *
length SMALLINT
replacement_cost DECIMAL(5,2) *
rating film_rating_enum
last_update DATETIME *
special_features TEXT
fulltext_col TEXT *
category
category_id INT * pk
name VARCHAR(25) *
last_update DATETIME *
country
country_id INT * pk
country VARCHAR(50) *
last_update DATETIME *
city
city_id INT * pk
city VARCHAR(50) *
country_id INT *
last_update DATETIME *
address
address_id INT * pk
address VARCHAR(50) *
address2 VARCHAR(50)
district VARCHAR(20) *
city_id INT *
postal_code VARCHAR(10)
phone VARCHAR(20) *
last_update DATETIME *
film_actor
actor_id INT *
film_id INT *
last_update DATETIME *
inventory
inventory_id INT * pk
film_id INT *
store_id INT *
last_update DATETIME *
film_category
film_id INT *
category_id INT *
last_update DATETIME *
customer
customer_id INT * pk
store_id INT *
first_name VARCHAR(45) *
last_name VARCHAR(45) *
email VARCHAR(50)
address_id INT *
activebool BOOLEAN *
create_date DATE *
last_update DATETIME
active INT
staff
staff_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
address_id INT *
email VARCHAR(50)
store_id INT *
active BOOLEAN *
username VARCHAR(16) *
password VARCHAR(40)
last_update DATETIME *
picture BLOB
rental
rental_id INT * pk
rental_date DATETIME *
inventory_id INT *
customer_id INT *
return_date DATETIME
staff_id INT *
last_update DATETIME *
store
store_id INT * pk
manager_staff_id INT *
address_id INT *
last_update DATETIME *
payment
payment_id INT * pk
customer_id INT *
staff_id INT *
rental_id INT
amount DECIMAL(5,2) *
payment_date DATETIME *